

Research Notes on Weight initialization

Weight initialization

>> Section 1

He initialization As Glorot initialization performs poorly for ReLU activation, He initialization (or Kaiming initialization) was proposed by Kaiming He et al. for networks with ReLU activation. It samples each entry in $W^{(l)}$ from $N(0, 2/n_{l-1})$.

>> Section 2

Glorot initialization Glorot initialization (or Xavier initialization) was proposed by Xavier Glorot and Yoshua Bengio. It was designed as a compromise between two goals: to preserve activation variance during the forward pass and to preserve gradient variance during the backward pass. For uniform initialization, it samples each entry in $W^{(l)}$ independently and identically from $U(\pm \sqrt{6/(n_{l+1} + n_{l-1})})$. In the context, n_{l-1} is also called the "fan-in", and n_{l+1} the "fan-out". When the fan-in and fan-out are equal, then Glorot initialization is the same as LeCun initialization.

>> Section 3

Fixup initialization In 2015, the introduction of residual connections allowed very deep neural networks to be trained, much deeper than the ~20 layers of the previous state of the art (such as the VGG-19). Residual connections gave rise to their own weight initialization problems and strategies. These are sometimes called "normalization-free" methods, since using residual connection could stabilize the training of a deep neural network so much that normalizations become unnecessary. Fixup initialization is designed specifically for networks with residual connections and without batch normalization, as follows:

>> Section 4

Others Instead of initializing all weights with random values on the order of $O(1/\sqrt{n})$, sparse initialization initialized only a small subset of the weights with larger random values, and the other weights zero, so that the total variance is still on the order of $O(1)$. Random walk initialization was designed for MLP so that during backpropagation, the L2 norm of gradient at each layer performs an unbiased random walk as one moves from the last layer to the first. Looks linear initialization was designed to allow the neural network to behave like a deep linear network at initialization, since $W \text{ReLU}(x) - W \text{ReLU}(-x) = Wx$. It initializes a matrix W of

shape $R^{n \times m}$ by any method, such as orthogonal initialization, then let the $R^{n \times m}$ weight matrix to be the concatenation of W , $-W$.

>> Section 5

Further reading Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron (2016). "8.4 Parameter Initialization Strategies". Deep learning. Adaptive computation and machine learning. Cambridge, Mass: The MIT press. ISBN 978-0-262-03561-3. Narkhede, Meenal V.; Bartakke, Prashant P.; Sutaone, Mukul S. (June 28, 2021). "A review on weight initialization strategies for neural networks". Artificial Intelligence Review. 55 (1). Springer Science and Business Media LLC: 291–322. doi:10.1007/s10462-021-10033-z. ISSN 0269-2821.

>> Section 6

Miscellaneous For hyperbolic tangent activation function, a particular scaling is sometimes used: $1.7159 \tanh(x/3)$. This was sometimes called "LeCun's tanh". It was designed so that it maps the interval $[-1, +1]$ to itself, thus ensuring that the overall gain is around 1 in "normal operating conditions", and that $|f'(x)|$ is at maximum when $x = -1, +1$, which improves convergence at the end of training. In self-normalizing neural networks, the SELU activation function $\text{SELU}(x) = \lambda \{ x \text{ if } x > 0, e^{-\alpha} x - \alpha \text{ if } x \leq 0 \}$ with parameters $\lambda \approx 1.0507$, $\alpha \approx 1.6733$ makes it such that the mean and variance of the output of each layer has $(0, 1)$ as an attracting fixed-point. This makes initialization less important, though they recommend initializing weights randomly with variance $1/n_{l-1}$.

>> Section 7

Orthogonal initialization (Saxe et al. 2013) proposed orthogonal initialization: initializing weight matrices as uniformly random (according to the Haar measure) semi-orthogonal matrices, multiplied by a factor that depends on the activation function of the layer. It was designed so that if one initializes a deep linear network this way, then its training time until convergence is independent of depth. Sampling a uniformly random semi-orthogonal matrix can be done by initializing X by IID sampling its entries from a standard normal distribution, then calculate $(XX^T)^{-1/2}X$ or its transpose, depending

Research Notes on Weight initialization

Weight initialization

on whether X is tall or wide. For CNN kernels with odd widths and heights, orthogonal initialization is done this way: initialize the central point by a semi-orthogonal matrix, and fill the other entries with zero. As an illustration, a kernel K of shape $3 \times 3 \times c \times c'$ is initialized by filling $K[2, 2, :, :]$ with the entries of a random semi-orthogonal matrix of shape $c \times c'$, and the other entries with zero. (Balduzzi et al., 2017) used it with stride 1 and zero-padding. This is sometimes called the Orthogonal Delta initialization. Related to this approach, unitary initialization proposes to parameterize the weight matrices to be unitary matrices, with the result that at initialization they are random unitary matrices (and throughout training, they remain unitary). This is found to improve long-sequence modelling in LSTM. Orthogonal initialization has been generalized to layer-sequential unit-variance (LSUV) initialization. It is a data-dependent initialization method, and can be used in convolutional neural networks. It first initializes weights of each convolution or fully connected layer with orthonormal matrices. Then, proceeding from the first to the last layer, it runs a forward pass on a random minibatch, and divides the layer's weights by the standard deviation of its output, so that its output has variance approximately 1.